Employee Management System

Code Review Guide

Spring Boot 3.x · Java 21 · REST API

Attribute	Value
Framework	Spring Boot 3.3.5 (target: 3.5.x)
Language	Java 21 (LTS)
Architecture	Layered MVC (Controller → Service → Repository)
Database	H2 In-Memory (JDBC, JPA/Hibernate)
Excel I/O	Apache POI 5.3.0 (XSSFWorkbook — xlsx only)
PDF	OpenPDF 1.3.43
Email	Spring Mail (async, dev-mock aware)
API Docs	SpringDoc OpenAPI 2.6.0 / Swagger UI
Test	JUnit 5 + Mockito + MockMvc

1. Architecture & Layering

The project follows strict **Clean Layered Architecture**. Each layer has a single responsibility and communicates only with the layer directly below it.

Layer	Class(es)	Responsibility
Presentation	EmployeeController	Parse HTTP, delegate to service, wrap in ApiResponse
Business	EmployeeServiceImpl	Enforce ALL business rules — email uniqueness, salary floors, soft-delete
Data Access	EmployeeRepository	JPA queries — no logic whatsoever
Cross-cutting	GlobalExceptionHandler EmailServiceImpl ExcelServiceImpl PdfServiceImpl	Handle errors, async email, file I/O

Key architectural rule: The controller contains ZERO business logic. It only translates HTTP requests into service calls and wraps results in ApiResponse.

The DTO Barrier

Entity classes are **never** returned directly from any endpoint. Instead, dedicated DTO classes act as a translation layer between the API contract and the internal domain model:

- EmployeeRequestDto — input DTO; annotated for Bean Validation
- EmployeePatchDto — partial update; only salary / department / active
- EmployeeResponseDto — output DTO; no JPA annotations leak out
- ApiResponse<T> — generic envelope wrapping every response

2. Generic Response Wrapper — ApiResponse<T>

Every endpoint returns **ApiResponse<T>** — a generic, immutable envelope ensuring every client always receives the same JSON structure:

```
{ "success": true, "status": 201, "message": "Employee created", "data": {...}, "timestamp": "..." }
```

The class uses static factory methods instead of constructors, making call-sites self-documenting:

```
// At controller level — intention is clear ApiResponse.created("Employee created successfully", dto) ApiResponse.ok("Employees retrieved", pagedResult) ApiResponse.error("Email already registered", 409)
```

@JsonInclude(NON_NULL) ensures null fields (e.g. data in error responses) are omitted from the JSON output, keeping payloads clean.

3. Exception Handling

GlobalExceptionHandler (annotated **@RestControllerAdvice**) is the single place that maps exceptions to HTTP responses. No try/catch appears in the controller.

Exception Class	HTTP Status	When Thrown
EmployeeNotFoundException	404 Not Found	ID not found in database
DuplicateEmailException	409 Conflict	Email already registered
InvalidFileFormatException	400 Bad Request	Non-.xlsx file uploaded
ExcelProcessingException	500 Internal Error	I/O / corrupt file error
MethodArgumentNotValidException	400 Bad Request	@Valid fails on @RequestBody
ConstraintViolationException	400 Bad Request	@Validated on service method
MaxUploadSizeExceededException	400 Bad Request	File exceeds 10 MB

```
Validation errors include per-field messages: { "success": false, "status": 400, "data": { "firstName": "First name is required" } }
```

The catch-all handler for **Exception.class** never leaks stack traces — it returns a generic "unexpected error" message and logs the full exception server-side.

4. Entity Design & JPA

The **Employee** entity is annotated with Lombok (@Getter, @Setter, @Builder) and Jakarta Validation constraints. All validation annotations come from **jakarta.validation.constraints** — never from Hibernate-specific packages (which would couple the model to the ORM).

Field	Type	Constraints	Notes
id	Long	@Id, @GeneratedValue(AUTO)	Auto-generated PK
firstName	String	@NotBlank, @Size(max=50)	
lastName	String	@NotBlank, @Size(max=50)	
email	String	@NotBlank, @Email, @Column(unique=true)	Unique across all records
department	String	@NotBlank, @Size(max=100)	
salary	BigDecimal	@NotNull, @DecimalMin("0.00")	Precision(15,2)
dateOfJoining	LocalDate	@NotNull, @PastOrPresent	ISO-8601 in JSON
active	Boolean	@NotNull, @Builder.Default=true	Soft-delete flag
createdAt	LocalDateTime	@Column(updatable=false)	Set in @PrePersist
updatedAt	LocalDateTime	—	Refreshed in @PreUpdate

Why BigDecimal for salary? Float and double use binary floating-point, which cannot exactly represent decimal numbers like 0.1. BigDecimal avoids rounding errors in financial calculations.

Audit timestamps are set via JPA lifecycle callbacks, not in business logic: @PrePersist — sets createdAt and updatedAt to LocalDateTime.now() @PreUpdate — refreshes updatedAt only

Soft Delete Pattern

Rather than physically removing records, DELETE sets **active=false**. This preserves audit history, maintains referential integrity, and allows recovery. A separate **hard-delete** endpoint exists but enforces a precondition: the record must already be inactive.

CORRECT — soft delete:

```
// Service layer employee.setActive(false); employeeRepository.save(employee); // Row stays in DB; email notification sent async
```

INCORRECT — removing the row on first delete:

```
// Never do this for a first DELETE call employeeRepository.delete(employee);
```

5. Service Layer — Business Rules

EmployeeServiceImpl is annotated **@Validated** so that **@Valid** on method parameters triggers Bean Validation through AOP interception, bubbling up as ConstraintViolationException.

Business Rule: Salary Floor by Department

validateSalaryFloor() enforces minimum salaries *before* any persistence occurs. These thresholds are externalised to application.properties so they can change without recompiling:

```
app.business.salary.floor.default=30000 # Engineering, HR, etc.
app.business.salary.floor.intern=15000 # Intern department only
```

The comparison uses BigDecimal.compareTo() — never == or .equals() for currency values.

Business Rule: Duplicate Email Guard

```
On CREATE: employeeRepository.existsByEmail(email) — simple existence check. On UPDATE:
existsByEmailAndIdNot(email, id) — allows the same employee to keep their email.
```

Transaction Boundaries

The service class is annotated **@Transactional(readOnly=true)** by default — read-only transactions allow the database to optimise queries. Write methods (**@Transactional** without readOnly) override this at the method level:

```
@Transactional(readOnly = true) // ← class default public class EmployeeServiceImpl {
@Transactional // ← overrides for writes public EmployeeResponseDto createEmployee(...) {
... }
```

6. Repository Layer

EmployeeRepository extends **JpaRepository<Employee, Long>**. Spring Data generates the full implementation at startup from method signatures — no implementation code is written.

Method	Type	Description
findByDepartment(dept)	Derived	WHERE department = :dept
findByEmail(email)	Derived	Returns Optional — safe for existence checks
findByActiveTrue()	Derived	WHERE active = true
existsByEmail(email)	Derived	Returns boolean — cheaper than findBy
existsByEmailAndIdNot(email, id)	Derived	Duplicate check excluding self
findByDepartmentAndActive(...)	Derived + Pageable	Paginated filter
findBySalaryRange(min, max)	@Query JPQL	BETWEEN query for salary filter
purgeInactiveEmployees()	@Query + @Modifying	DELETE WHERE active = false

No SQL strings in the service layer. All data access goes through the repository. If a query cannot be expressed by method naming, use @Query with JPQL (or native SQL only as a last resort).

7. Excel Import — Apache POI

ExcelServiceImpl uses **XSSFWorkbook** exclusively — the modern OOXML (.xlsx) format. HSSFWorkbook (.xls) is intentionally unsupported and rejected with HTTP 400.

Import Processing Pipeline

1. **Validate file extension:** Check filename ends with .xlsx; throw InvalidFileFormatException otherwise.
2. **Open with try-with-resources:** XSSFWorkbook is AutoCloseable — always use try-with-resources to prevent memory leaks.
3. **Skip header row:** Row index 0 is always the header. Data starts at index 1.
4. **Extract cells via CellExtractorUtil:** Normalises all cell types (STRING, NUMERIC, BOOLEAN, FORMULA) to Java types.
5. **Build EmployeeRequestDto:** Each row becomes a DTO — same DTO used by the REST endpoint.
6. **Bean Validation via Validator:** Validator.validate(dto) — collects ALL violations without fail-fast.
7. **Aggregate row errors:** Errors stored as "Row N: field – message". Failed rows are skipped, not aborted.
8. **Persist valid rows:** employeeRepository.save() inside the same transaction.
9. **Return ImportResultDto:** successCount, failureCount, List errors returned to client.

Transaction boundary: The entire import runs in one @Transactional. Validation failures skip the row (no rollback). System errors (IOException, DB failure) trigger a full rollback of all rows processed so far.

CellExtractorUtil

A utility class that safely reads typed values from POI Cell objects, handling all CellType variants (STRING, NUMERIC, BOOLEAN, FORMULA, BLANK). This prevents NullPointerExceptions and type-mismatch exceptions in the import loop.

```
// Extract a string from any cell type String value =
CellExtractorUtil.getString(row.getCell(COL_EMAIL)); // Extract double (handles numeric and
string cells) double salary = CellExtractorUtil.getDouble(row.getCell(COL_SALARY));
```

8. Excel Export

The export streams the workbook directly to **HttpServletResponse.getOutputStream()** — no temp files are created on disk. The filename includes a timestamp to prevent browser caching.

- **Content-Type:** application/vnd.openxmlformats-officedocument.spreadsheetml.sheet
- **Content-Disposition:** attachment; filename="employees_20240115_103045.xlsx"
- **Sheet name:** "Employees"
- **Row 0:** Bold header with dark-blue background, white text
- **Alternating rows:** White / light-blue fill for readability
- **Salary cells:** Numeric format #,##0.00; right-aligned
- **Date cells:** ISO string format YYYY-MM-DD
- **Auto-size:** sheet.autoSizeColumn(i) called for every column

9. PDF Report Generation — OpenPDF

PdfServiceImpl uses **OpenPDF** (a liberally licensed fork of iText 4). The report streams directly to the response output stream — no temp files.

PDF Features

- Title header: company name, report title, generated timestamp, total record count
- Data table: 7 columns — ID, Full Name, Email, Department, Salary, Date of Joining, Status
- Alternating row colours (white / light-blue)
- Bold column headers with dark-blue background
- Salary column: right-aligned with $ symbol and 2 decimal places
- Inactive employees: rendered in grey with STRIKETHRU font style
- Page footer: "Page N of M" on every page using PdfTemplate for the total pages count

"Page N of M" Implementation

The total page count is unknown when Page 1 is rendered. OpenPDF solves this with a **PdfTemplate** — a placeholder written on every page that is filled in during onCloseDocument() once the total is known:

```
// onOpenDocument: create placeholder totalPagesTemplate =
writer.getDirectContent().createTemplate(30, 12); // onEndPage: draw "Page N of "
cb.showText("Page " + writer.getPageNumber() + " of "); cb.addTemplate(totalPagesTemplate, x,
y); // placeholder // onCloseDocument: fill placeholder with final total
totalPagesTemplate.showText(String.valueOf(writer.getPageNumber() - 1));
```

10. Email Service — Async Notifications

EmailServiceImpl is annotated with **@Async("taskExecutor")** on every public method. This means email calls run on a separate thread pool thread, completely decoupling email latency from the HTTP response time.

Notification Events

Event	Method	Recipient
Employee created	sendWelcomeEmail()	New employee's email
Soft delete	sendDeactivationEmail()	Deactivated employee's email
Excel import complete	sendImportSummaryEmail()	System admin address

Dev Mode (app.email.enabled=false)

When email is disabled (the default for local development), all email content is logged at INFO level with [EMAIL-MOCK] prefix. No SMTP connection is made. The application starts cleanly even with no mail server configured.

> MailException is always caught inside the send() method — email failure MUST NOT propagate, as it would roll back the enclosing database transaction.

11. Best Practices Checklist

Practice	Implementation	Status
No wildcard imports	Every import is explicit — no "import java.util.*"	✓
Generic ApiResponse<T>	All endpoints return the same JSON envelope	✓
DTO layer	Entity never exposed via API — DTO in / DTO out	✓
Constructor injection	Dependencies injected via constructor, not @Autowired field	✓
@Transactional(readOnly=true)	Default on service class; write methods override	✓
Soft delete	DELETE sets active=false; record retained for audit	✓
Business rules in service	Controller is thin — zero logic	✓
Async email	@Async prevents email latency blocking HTTP responses	✓
Email failure isolation	MailException caught — never rolls back DB transaction	✓
try-with-resources	XSSFWorkbook always closed; no resource leaks	✓
No temp files	Excel and PDF streamed directly to response output stream	✓
Per-row import errors	Failed rows skip without aborting entire import	✓
Configurable thresholds	Salary floors in application.properties — no magic numbers	✓
Logging with SLF4J	INFO for business events, DEBUG for internals — no System.out.println	✓
JavaDoc on interfaces	EmployeeService fully documented — single source of truth	✓

12. Common Pitfalls & How This Project Avoids Them

Exposing JPA entities directly via REST

Problem: @GetMapping("/{id}") returns Employee entity — Jackson serialises lazy proxies, triggers N+1 queries, leaks internal structure.

Solution: Always map to EmployeeResponseDto before returning. The entity stays inside the service boundary.

Business logic in the controller

Problem: if (employeeRepository.existsByEmail(...)) { throw ...; } — placed in the @PostMapping method.

Solution: The controller calls employeeService.createEmployee(dto) and nothing else. All guards are inside EmployeeServiceImpl.

Hardcoding salary floors

Problem: if (salary.compareTo(new BigDecimal("30000")) < 0) — magic number, not testable without code change.

Solution: @Value("${app.business.salary.floor.default}") BigDecimal salaryFloorDefault — override in tests or per-env.

Using float/double for money

Problem: double salary = 19.90; double tax = salary * 0.1; // → 1.9899999... (floating-point error)

Solution: BigDecimal salary with scale(2, HALF_UP) — exact decimal arithmetic guaranteed.

Forgetting to close the POI Workbook

Problem: XSSFWorkbook workbook = new XSSFWorkbook(file.getInputStream()); /* no close */ — memory and file descriptor leak.

Solution: try (XSSFWorkbook workbook = ...) { } — AutoCloseable ensures close() on every exit path.

Email failure rolling back DB transaction

Problem: mailSender.send() throws MailException inside a @Transactional method — Spring rolls back the employee save.

Solution: Email is @Async + wrapped in try/catch(MailException). If it fails, it logs and returns — transaction is unaffected.

13. Testing Strategy

@WebMvcTest — Controller Layer

@WebMvcTest(EmployeeController.class) loads only the web layer — no database, no service beans. Services are mocked with @MockBean. MockMvc drives HTTP requests.

```
What is tested: • Correct HTTP status codes (201, 200, 204, 404, 409...) • Input validation
(@Valid failures return 400 with field messages) • Exception handler mapping
(EmployeeNotFoundException → 404) • Response JSON structure (ApiResponse envelope)
```

@ExtendWith(MockitoExtension) — Service Layer

Pure unit tests: no Spring context, no database. Mockito stubs the repository. ReflectionTestUtils.setField() injects @Value fields that Spring would normally provide.

```
What is tested: • Business rules: salary floors, duplicate email guard • Soft vs hard delete
rules • EmployeeNotFoundException thrown when ID not found • mapToResponseDto correctness
```

@SpringBootTest — Integration

EmployeeManagementApplicationTests verifies the full context loads successfully. Uses a separate application.properties in src/test/resources with email disabled and Swagger docs turned off for speed.

14. How to Run & Verify

Start the application:

```
./mvnw spring-boot:run
```

What to verify	URL / Command
Swagger UI (explore all endpoints)	http://localhost:8080/swagger-ui.html
H2 Database Console	http://localhost:8080/h2-console
Create employee (curl)	curl -X POST ... (see README for full example)
Export Excel	curl -o out.xlsx http://localhost:8080/api/v1/employees/export/excel
Export PDF	curl -o out.pdf http://localhost:8080/api/v1/employees/export/pdf
Import Excel	curl -X POST -F file=@template.xlsx .../import
Salary range filter	GET /api/v1/employees/salary-range?min=50000&max;=100000
Application log	tail -f logs/employee-management.log
Run all tests	./mvnw test

This guide covers the complete implementation. For endpoint details, use the Swagger UI at /swagger-ui.html which documents every request/response schema interactively.